

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Reidita Eirene Anastasia Katampuge - 5024241001

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (*Blink*)

1. Mengisi `platformio.ini` dengan *platform* `ststm32`, *board* `blackpill_f401cc`, *framework* `arduino`, serta `upload_protocol` dan `debug_tool` bernilai `stlink`.

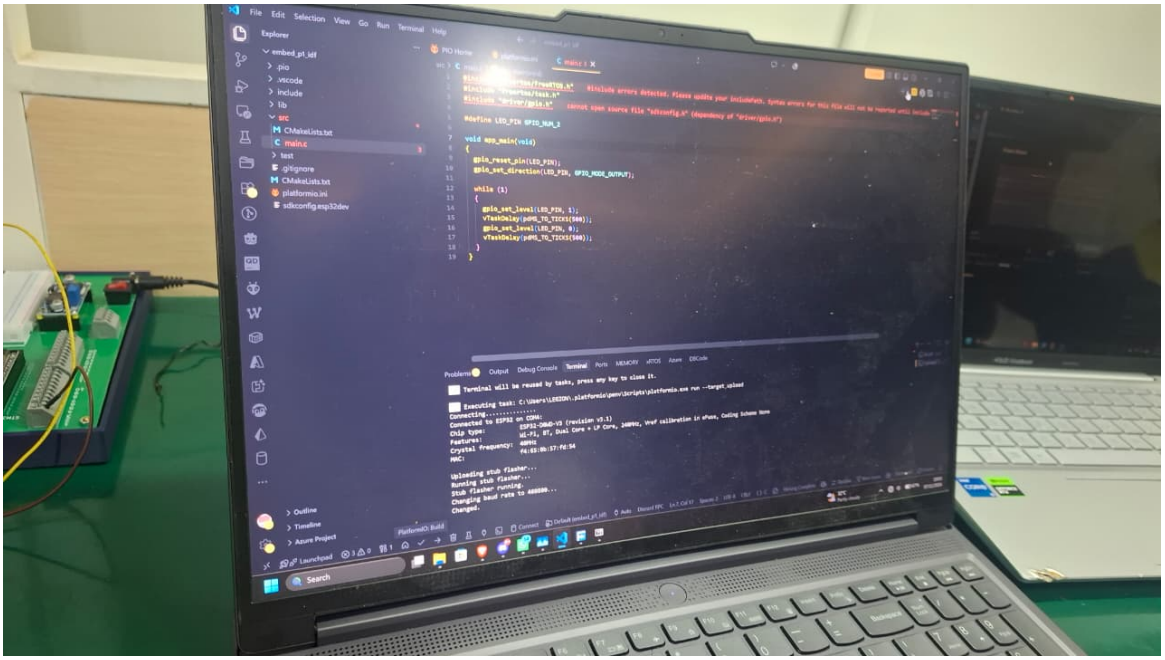
2. Menghubungkan ST-Link ke Blackpill dengan kabel jumper sesuai pasangan pin SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3, dengan memastikan ST-Link memakai tegangan 3,3 V. Setelah itu, ST-Link dihubungkan ke komputer melalui kabel USB.



Gambar 1: Hubungkan ST-Link ke Blackpill lalu ST-Link dihubungkan ke komputer

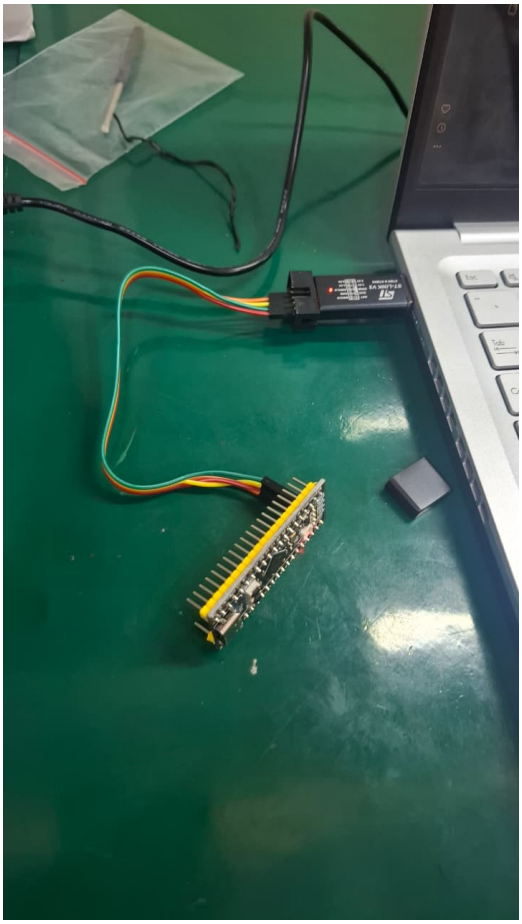
3. Menulis program *Blink* pada `src/main.cpp` yang mengatur pin PC13 sebagai keluaran dan menyalakan serta mematikan LED setiap 500 ms. LED *onboard* bersifat aktif *LOW*.

4. Menekan *Build* lalu *Upload*.



Gambar 2: Menekan Build dan Upload

5. Mengamati LED *onboard* pada pin PC13, yang seharusnya menyala 0,5 detik dan mati 0,5 detik secara berulang.



Gambar 3: Amati LED

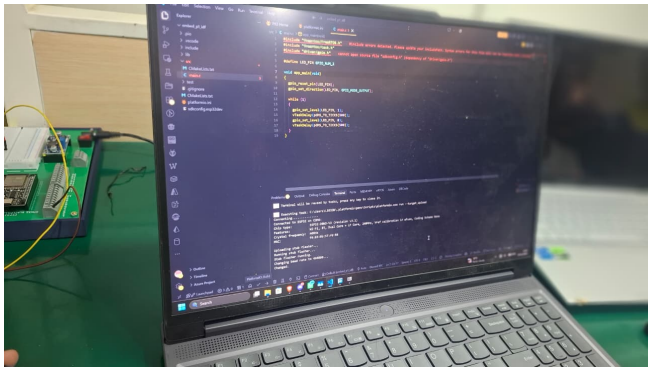
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (*Blink*)

1. Mengganti isi `platformio.ini` dengan *platform* `espressif32`, *board* `esp32dev`, dan *framework* `espidf`, lalu menunggu proses *refresh* selesai.
2. Menghubungkan ESP32 ke komputer melalui kabel USB dan memastikan port-nya dikenali.



Gambar 4: Hubungkan ESP32 ke Komputer

3. Menghapus berkas `src/main.cpp`, kemudian membuat berkas baru `src/main.c`.
4. Menulis program *Blink* dengan fungsi `app_main()` yang mengatur GPIO 2 sebagai keluaran dan mengubah levelnya setiap 500 ms menggunakan `vTaskDelay(pdMS_TO_TICKS(500))`.
5. Menekan *Build* lalu *Upload*



Gambar 5: Build Lalu Upload

6. Mengamati LED onboard ESP32.

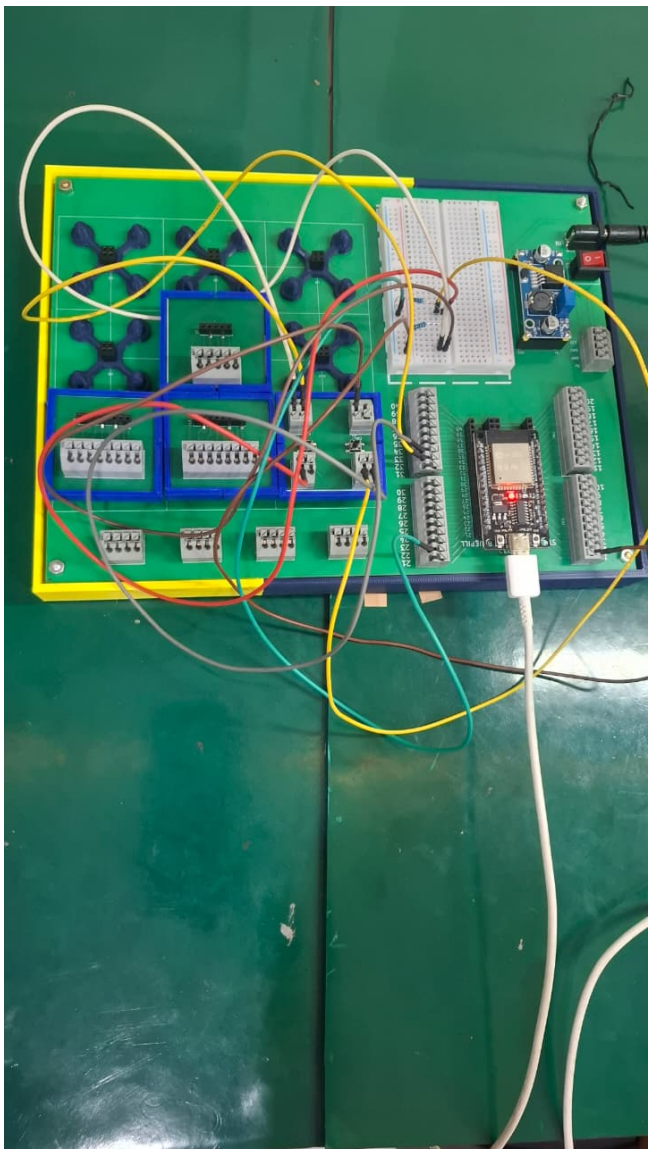


Gambar 6: Mengamati LED ESP32

1.3 Percobaan 3: *Pull-up* dan *Pull-down* Eksternal dan Internal

Bagian A: resistor eksternal

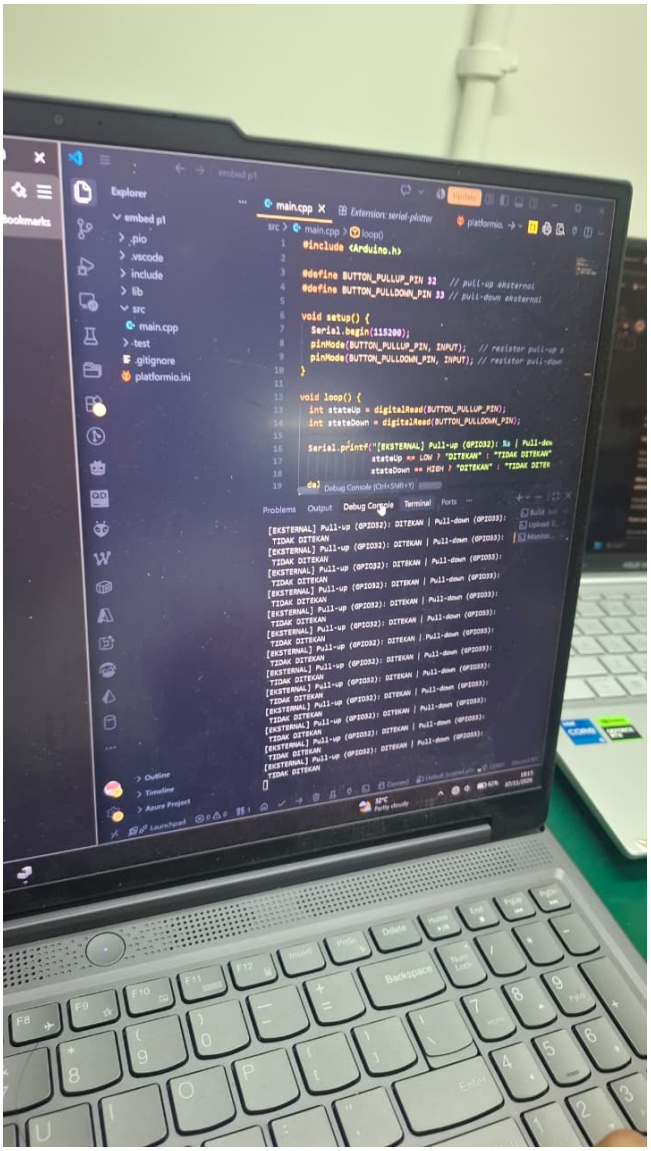
1. Mengganti isi `platformio.ini` menjadi *platform* `espressif32`, *board* `esp32dev`, dan *framework* `arduino`. Berkas `src/main.c` dihapus dan `src/main.cpp` dibuat kembali.
2. Merangkai tombol 1 pada GPIO 32 dengan resistor 10 kΩ ke 3,3 V sebagai *pull-up*, dan kaki tombol lainnya ke GND.
3. Merangkai tombol 2 pada GPIO 33 dengan resistor 10 kΩ ke GND sebagai *pull-down*, dan kaki tombol lainnya ke 3,3 V.



Gambar 7: Merangkai Rangkaian

4. Menulis Kode Program A, yang mengatur kedua pin sebagai `INPUT` biasa dan mencetak status tombol ke *Serial Monitor* setiap 200 ms.

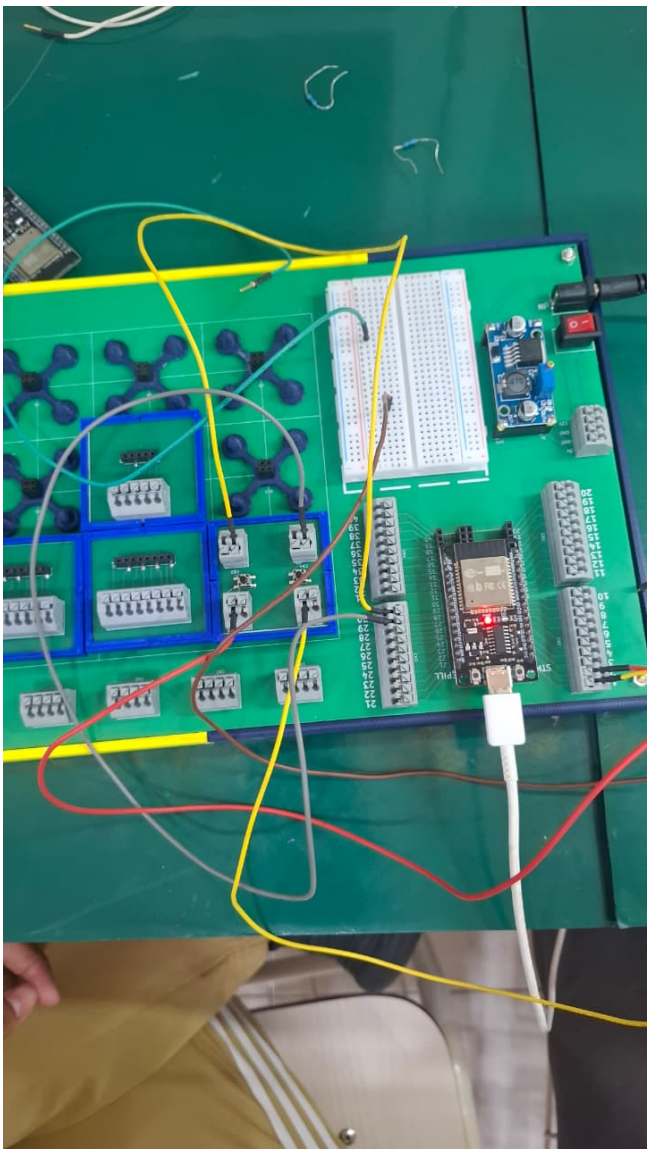
5. Menekan *Build* dan *Upload*, membuka *Serial Monitor* pada 115200 *baud*, menekan kedua tombol, lalu mencatat hasilnya.



Gambar 8: Output Pada Serial Monitor

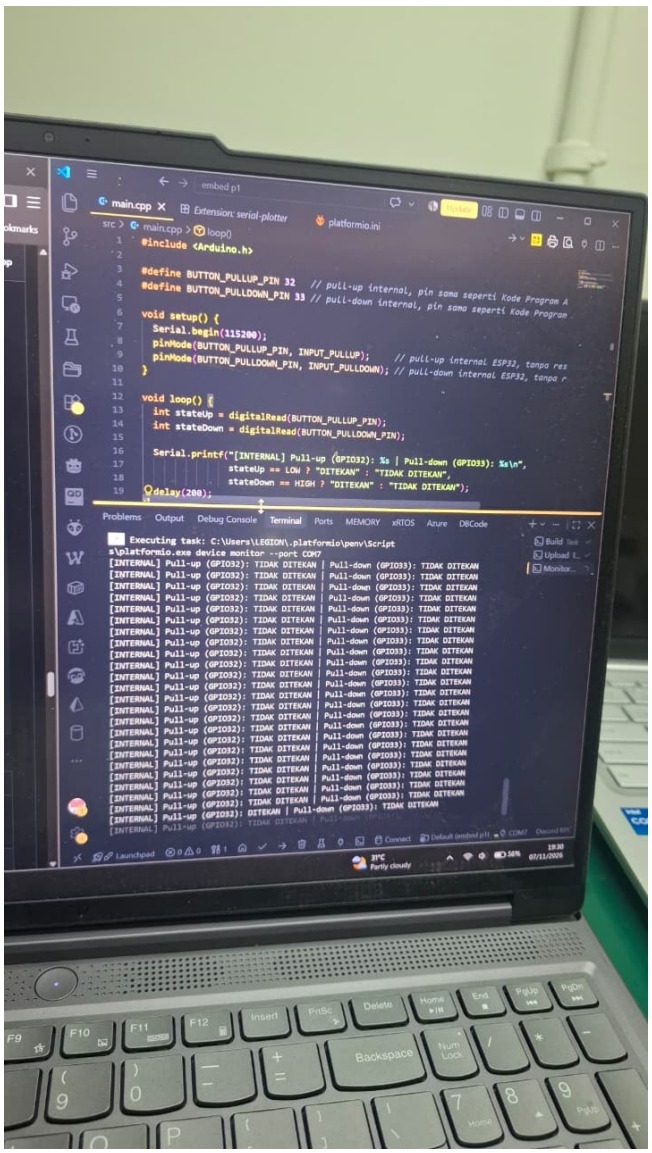
Bagian B: resistor internal

6. Melepas kedua resistor eksternal dengan tombol tetap terpasang pada GPIO 32 (kaki lain ke GND) dan GPIO 33 (kaki lain ke 3,3 V), dengan memastikan pasangan kaki tombol yang dipakai benar.



Gambar 9: Lepas Resistor dalam Rangkaian

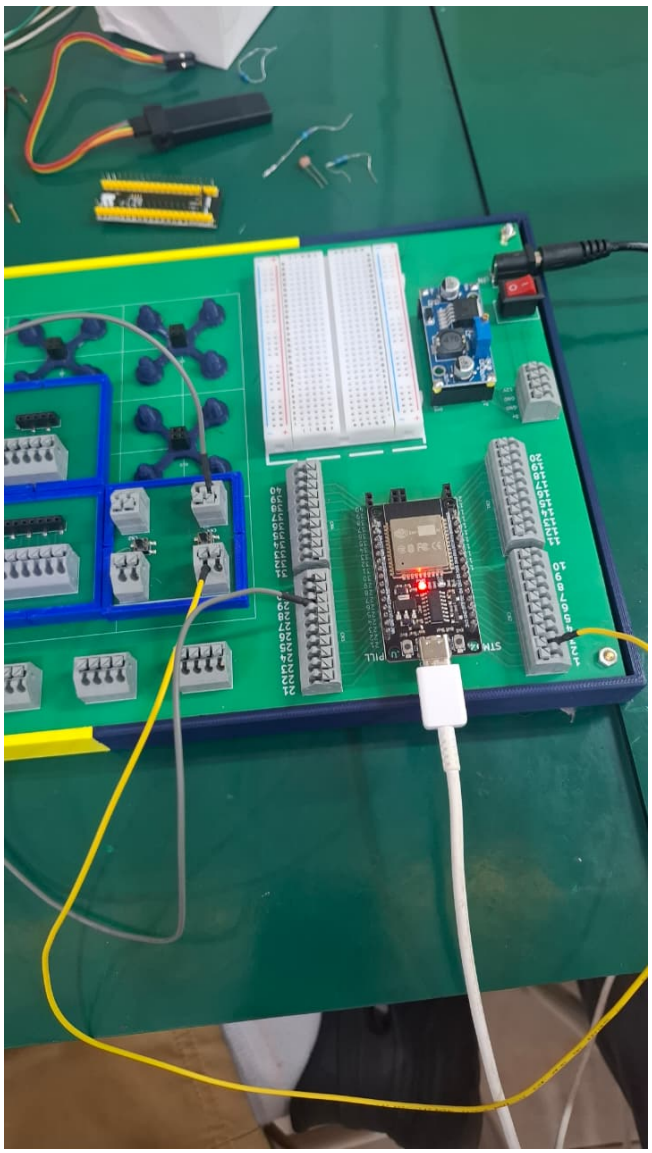
7. Mengganti isi `src/main.cpp` dengan Kode Program B, yang mengatur GPIO 32 sebagai `INPUT_PULLUP` dan GPIO 33 sebagai `INPUT_PULLDOWN`.
8. Menekan *Build* dan *Upload*, menekan kedua tombol yang sama, lalu mencatat hasilnya.
9. Membandingkan hasil Bagian A dan Bagian B. Hasil yang diharapkan sama, yaitu *pull-up* bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan *pull-down* sebaliknya. Bila hasilnya berbeda, diperiksa apakah kedua resistor eksternal sudah benar-benar dilepas.



Gambar 10: Output Serial Monitor

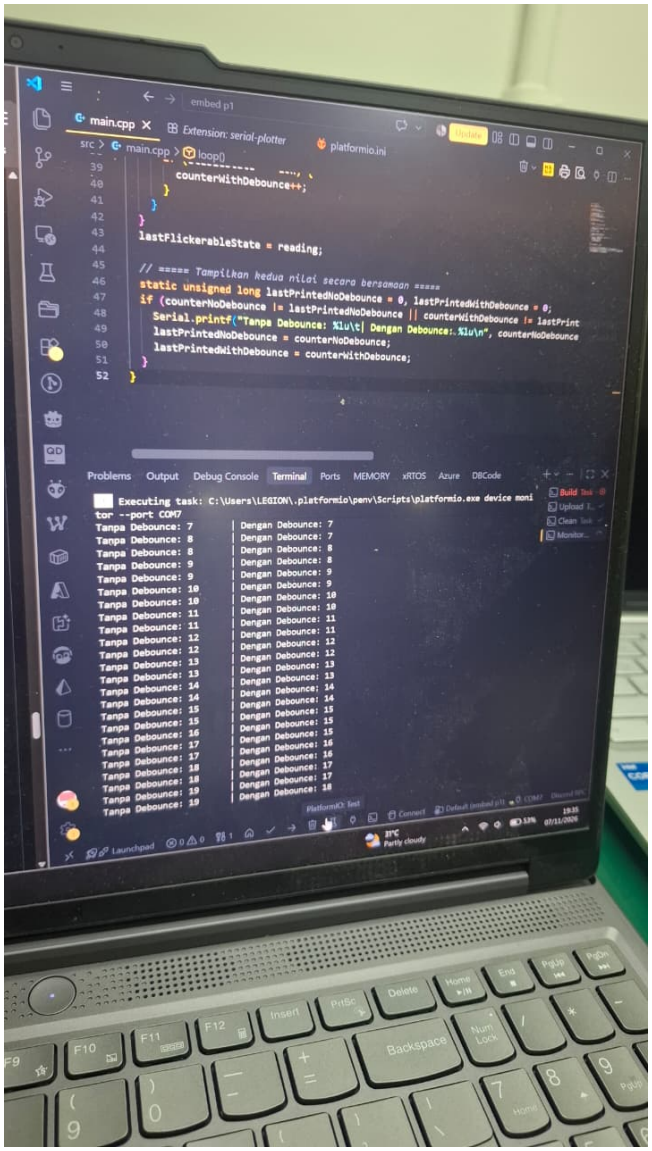
1.4 Percobaan 4: *Debouncing* pada Input Tombol GPIO

1. Memasang tombol pada GPIO 25, dengan satu kaki ke GPIO dan kaki lainnya ke GND. Pin dikonfigurasi dengan `INPUT_PULLUP`. Isi `platformio.ini` sama dengan Percobaan 3.



Gambar 11: Pasang Tombol pada GPIO 25, dengan Satu Kaki ke GPIO dan Kaki Lainnya ke GND

2. Menulis program dengan dua penghitung, yaitu `counterNoDebounce` (tanpa penyaring) dan `counterWithDebo` (dengan penyaring berbasis `millis()`), dengan `DEBOUNCE_DELAY` awal 50 ms.
3. Menekan *Build* dan *Upload*, lalu membuka *Serial Monitor* pada 115200 *baud*.

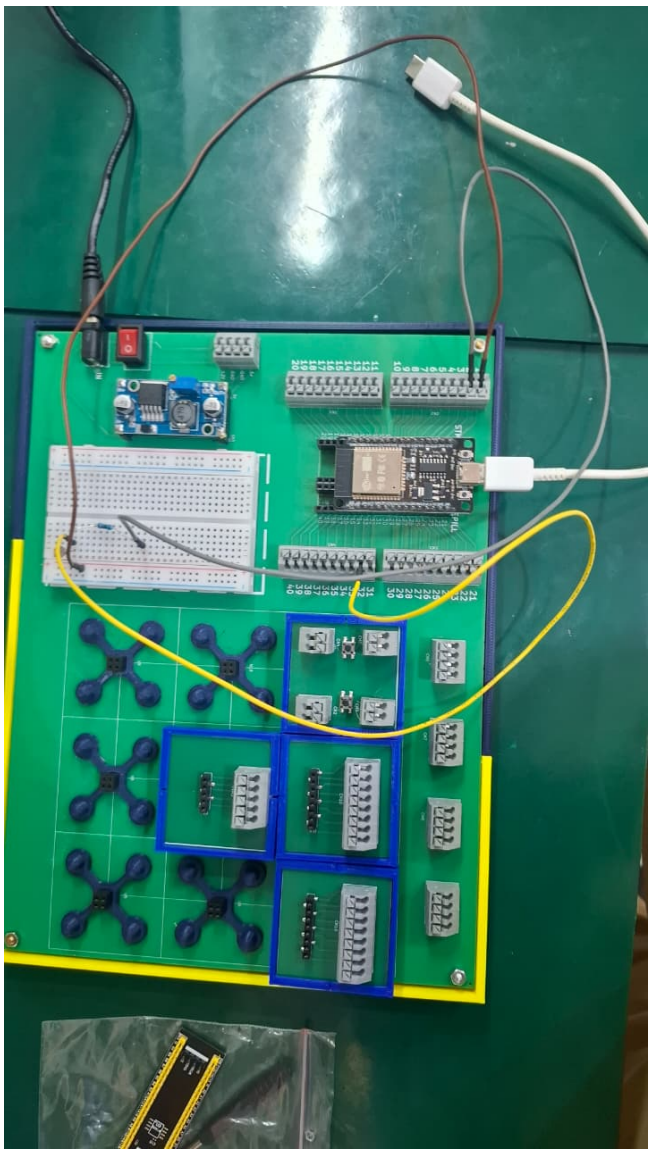


Gambar 12: Build Upload lalu Buka Serial Monitor

4. Menekan tombol satu kali, kemudian membandingkan nilai kedua penghitung.
5. Mengubah `DEBOUNCE_DELAY` menjadi 10 ms, mengunggah ulang program, lalu menekan tombol kembali dan mencatat hasilnya.
6. Mengubah `DEBOUNCE_DELAY` menjadi 200 ms, mengunggah ulang program, lalu menekan tombol dengan cepat berulang kali dan mencatat hasilnya.
7. Membandingkan hasil untuk 10 ms, 50 ms, dan 200 ms, lalu menganalisis kelebihan dan kekurangan tiap nilai.

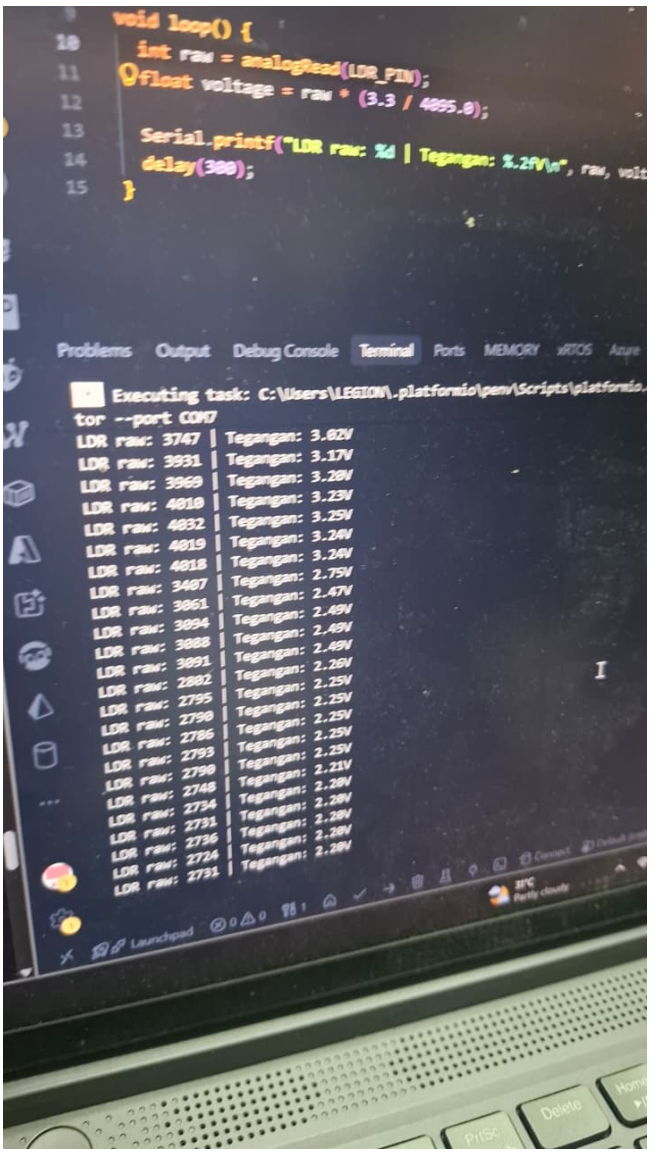
1.5 Percobaan 5: Pembacaan ADC dengan LDR

1. Merangkai pembagi tegangan dengan satu kaki LDR ke 3,3 V dan kaki lainnya ke GPIO 34 serta ke GND melalui resistor 10 kΩ. Isi `platformio.ini` sama dengan Percobaan 3.



Gambar 13: Merangkai Rangkaian

2. Menulis program yang membaca nilai ADC mentah dengan `analogRead()` pada GPIO 34, mengonversinya menjadi tegangan dengan $V = \text{raw} \times 3{,}3/4095$, dan mencetaknya ke *Serial Monitor* setiap 300 ms.
3. Menekan *Build* dan *Upload*, lalu membuka *Serial Monitor* pada 115200 *baud*.
4. Menutup LDR dengan tangan, kemudian menyinarinya dengan lampu, sambil mencatat perubahan nilai mentah (rentang 0 sampai 4095) dan tegangannya.



Gambar 14: Output dari LDR dari Serial Monitor

2 Analisis Hasil Percobaan

Pada Percobaan 1, LED *onboard* PC13 STM32 Blackpill berkedip dengan periode 1 detik (0,5 detik menyala, 0,5 detik padam). Hasil ini sesuai teori karena LED bersifat aktif *LOW*, sehingga menyala saat pin diberi LOW. Unggah melalui ST-Link V2 (SWD) berhasil karena sambungan SWDIO, SWCLK, GND, dan 3,3 V sudah benar. Jika SWDIO dan SWCLK tertukar, ST-Link terdeteksi tetapi *target* tidak terhubung.

Pada Percobaan 2, unggah program ESP-IDF berhasil. ESP32-D0WD-V3 terdeteksi di COM4 dan *baud rate* naik menjadi 460800, sehingga jalur USB-to-Serial (CP2102) bekerja dan tidak diperlukan *programmer* eksternal seperti pada Blackpill. Peringatan *"include errors detected"* berasal dari IntelliSense karena `sdkconfig.h` baru dibuat saat *build*, bukan kesalahan kompilasi. Program berjalan sebagai *task* FreeRTOS lewat `app_main()`, dan `vTaskDelay()` menyerahkan CPU ke *task* lain saat menunggu, berbeda dengan `delay()` pada Arduino.

Pada Percobaan 3 dengan resistor eksternal, *pull-up* (GPIO 32) terbaca *DITEKAN* dan *pull-down* (GPIO 33) terbaca *TIDAK DITEKAN*. Resistor 10 kΩ menarik GPIO 32 ke HIGH dan GPIO 33 ke LOW

saat tombol dilepas, sehingga pembacaan stabil dan tidak *floating*. Pada *pull-up*, tombol yang ditekan menghubungkan pin ke GND sehingga terbaca LOW (*DITEKAN*).

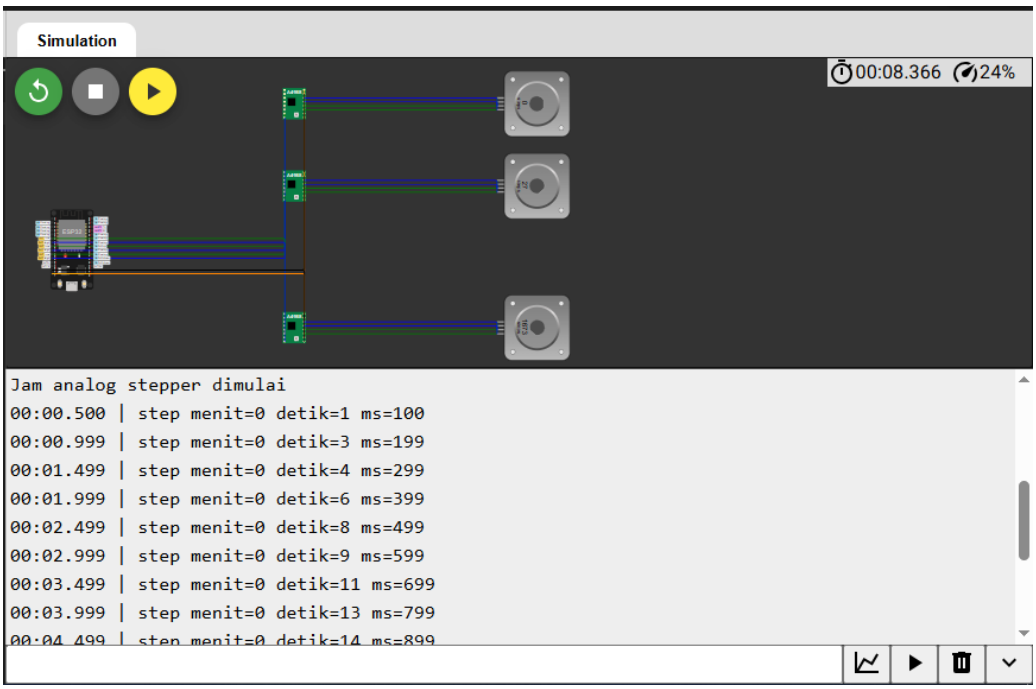
Pada Percobaan 3 tanpa resistor (internal), kedua tombol terbaca *TIDAK DITEKAN* secara stabil walaupun resistor eksternal dilepas, dan *pull-up* berubah menjadi *DITEKAN* saat tombol ditekan. Hal ini menunjukkan `INPUT_PULLUP` dan `INPUT_PULLDOWN` bekerja sama seperti resistor eksternal secara logika. Bedanya, resistor internal ESP32 sekitar 45 kΩ, lebih lemah daripada 10 kΩ sehingga lebih rentan derau, dan tidak tersedia pada GPIO 34–39. Hasil dapat menyimpang jika resistor eksternal belum dilepas atau pasangan kaki tombol salah.

Pada Percobaan 4, `counterNoDebounce` dan `counterWithDebounce` (50 ms) naik dengan jumlah yang sama pada tiap penekanan, sehingga *bouncing* tidak tampak pada tombol yang dipakai. Hal ini wajar karena getaran kontak tidak selalu muncul dan bergantung pada kualitas tombol serta cara menekan. Secara teori, 10 ms responsif tetapi dapat meloloskan *bouncing* yang lebih lama, sedangkan 200 ms bersih tetapi lambat dan dapat melewatkan penekanan cepat. Nilai 50 ms merupakan kompromi yang umum.

Pada Percobaan 5, nilai ADC turun dari sekitar 4030 (3,25 V) ke sekitar 2730 (2,20 V) seiring berkurangnya cahaya. Hasil ini sesuai teori pembagi tegangan, karena makin terang hambatan LDR makin kecil dan tegangan pada GPIO 34 makin besar. Konversinya benar, misalnya $2731 \times 3{,}3/4095 \approx 2{,}20$ V. ADC 12 bit memiliki 4096 tingkat (sekitar 0,8 mV per langkah) sehingga dapat merepresentasikan banyak kondisi, tidak hanya HIGH dan LOW. Nilai maksimum tidak mencapai 4095 karena ADC ESP32 tidak linier di ujung rentang, dan fluktuasi kecil (2790 sampai 2793) berasal dari derau ADC dan cahaya sekitar.

3 Hasil Tugas Modul

Tugas modul berupa simulasi jam analog dengan tiga motor *stepper* pada ESP32 di Wokwi. Setiap motor dikendalikan lewat *driver* A4988 dengan sinyal STEP dan DIR. Motor pertama menunjukkan menit (STEP GPIO 25, DIR GPIO 26), motor kedua detik (GPIO 27 dan 14), dan motor ketiga milidetik (GPIO 32 dan 33). Setiap motor memiliki 200 langkah per putaran (1,8° per langkah). Posisi tiap jarum dihitung dari waktu berjalan pada `micros()`, yaitu langkah target = waktu berjalan $\div$ periode jarum $\times$ 200, sehingga posisi tidak bergeser (*drift*). Hasil pada *Serial Monitor* sesuai perhitungan. Pada detik ke-6,499, jarum milidetik telah menempuh 1299 langkah (teori $\approx$ 1300), jarum detik 21 langkah (teori $\approx$ 21,6), dan jarum menit 0 langkah (teori $\approx$ 0,36, langkah pertamanya baru terjadi sekitar detik ke-18). Tangkapan layar simulasi ditunjukkan pada Gambar 15, dan *source code* program ditunjukkan pada Kode 1.



Gambar 15: Simulasi jam analog tiga motor *stepper* pada Wokwi

Listing 1: *Source code* jam analog tiga motor *stepper*

```
#include <Arduino.h>

const int TIME_SCALE = 1;
const long STEPS_PER_REV = 200;

struct Hand {
  const char* name;
  int stepPin;
  int dirPin;
  uint64_t periodUs;
  long pos;
};

Hand hands[3] = {
  {"menit",      25, 26, 3600000000ULL, 0},
  {"detik",      27, 14,   60000000ULL, 0},
  {"milidetik", 32, 33,    1000000ULL, 0}
};

void stepOnce(const Hand& h) {
  digitalWrite(h.stepPin, HIGH);
  delayMicroseconds(5);
  digitalWrite(h.stepPin, LOW);
  delayMicroseconds(5);
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(hands[i].stepPin, OUTPUT);
    pinMode(hands[i].dirPin, OUTPUT);
```

```
32      digitalWrite(hands[i].dirPin, HIGH); // searah jarum jam
33      digitalWrite(hands[i].stepPin, LOW);
34    }
35    Serial.println("Jam analog stepper dimulai");
36  }
37
38  void loop() {
39    uint64_t elapsedUs = (uint64_t)micros() * TIME_SCALE;
40
41    for (int i = 0; i < 3; i++) {
42      long target = (long)((elapsedUs * STEPS_PER_REV) / hands[i].periodUs);
43      int guard = 0;
44      while (hands[i].pos < target && guard < 20) {
45        stepOnce(hands[i]);
46        hands[i].pos++;
47        guard++;
48      }
49    }
50
51    static uint32_t lastPrint = 0;
52    if (millis() - lastPrint >= 500) {
53      lastPrint = millis();
54      uint64_t totalMs = elapsedUs / 1000ULL;
55      unsigned mm = (totalMs / 60000ULL) % 60;
56      unsigned ss = (totalMs / 1000ULL) % 60;
57      unsigned ms = totalMs % 1000ULL;
58      Serial.printf("%02u:%02u.%03u | step menit=%ld detik=%ld ms=%ld\n",
59                    mm, ss, ms, hands[0].pos, hands[1].pos, hands[2].pos);
60    }
61  }
```

Program mendefinisikan setiap jarum sebagai struktur `Hand` yang menyimpan pin STEP, pin DIR, periode satu putaran (`periodUs`), dan jumlah langkah yang sudah dijalankan (`pos`). Pada `loop()`, langkah target dihitung dari waktu berjalan, lalu jarum digerakkan sampai `pos` menyamai target. Batas `guard < 20` mencegah satu jarum memonopoli CPU sehingga ketiga jarum tetap dilayani bergantian. Nilai `TIME_SCALE` sama dengan 1 berarti waktu simulasi mengikuti waktu sebenarnya.

1. Kecepatan putar tiap jarum ditentukan dari waktu satu putaran penuh (periode T), dengan RPM $= 60/T$ untuk T dalam detik. Jarum milidetik berputar sekali per 1 detik sehingga 60 RPM, jarum detik sekali per 60 detik sehingga 1 RPM, dan jarum menit sekali per 3600 detik sehingga $\frac{1}{60}$ RPM. Jumlah langkah per putaran N mengikuti spesifikasi motor, yaitu 200 untuk motor 1,8° (atau 4096 untuk 28BYJ-48 dengan *gearbox*). Dari situ, frekuensi langkah $f = N/T$ dan jeda antarlangkah $t = T/N$. Pada $N = 200$, jarum milidetik memerlukan 200 langkah per detik (jeda 5 ms), jarum detik sekitar 3,33 langkah per detik (jeda 0,3 detik), dan jarum menit sekitar 0,0556 langkah per detik (jeda 18 detik). Agar jarum tetap akurat, program menghitung target langkah dari waktu absolut, bukan hanya mengandalkan `delay()` yang bisa menimbulkan galat kumulatif.

2. Motor *stepper* pada simulasi ini kurang tepat bila diganti motor servo. Servo standar hanya berputar sekitar 0–180°, sedangkan jarum jam harus berputar 360° secara terus-menerus. Selain itu, servo bergerak ke sudut tujuan secepat kemampuannya, sehingga kecepatan putarnya ti-

dak dapat diatur dengan presisi seperti pada *stepper* lewat jeda langkah. Jarum milidetik yang berputar 60 RPM juga tidak realistis bagi servo standar, dan jarum menit yang bergerak sangat lambat akan terhalang resolusi sudut servo. Servo putar kontinu (*continuous rotation*) memang dapat berputar penuh, tetapi kehilangan kemampuan menahan posisi sudut, sehingga posisi jarum tidak dapat dikendalikan. Servo hanya cocok bila tampilan jam diubah menjadi busur 180°.

3. *Driver* berfungsi sebagai penguat daya dan penerjemah sinyal antara mikrokontroler dan motor. Pin GPIO ESP32 hanya mengalirkan arus puluhan miliampere pada 3,3 V, sedangkan motor memerlukan arus dan tegangan jauh lebih besar yang tidak boleh diambil dari pin GPIO. *Driver* menerima sinyal logika berdaya rendah dari ESP32 dan menyalurkan daya dari catu eksternal ke motor. Pada A4988, *driver* juga menangani urutan pengaktifan koil, sehingga ESP32 cukup mengirim pulsa STEP dan level DIR. Pada motor DC, *driver* H-bridge (L298N) mengatur arah putar, dan pada motor *brushless*, ESC melakukan komutasi tiga fasa. *Driver* juga melindungi ESP32 dari arus balik (*back-EMF*) yang dihasilkan kumparan motor. Karena itu setiap motor pada percobaan harus memakai *driver*.

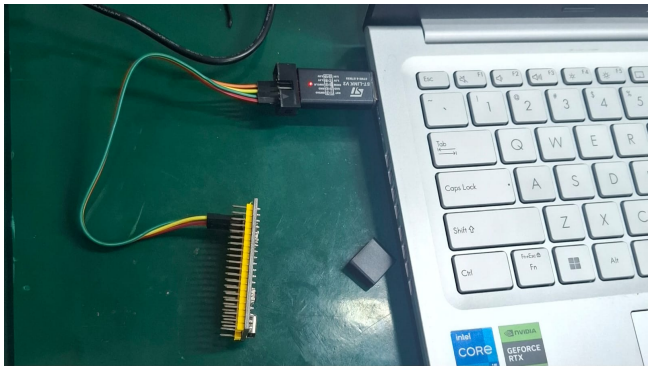
4 Kesimpulan

Praktikum ini menunjukkan bahwa mikrokontroler dapat diprogram dengan kerangka kerja dan jalur unggah yang berbeda sesuai papannya. STM32 Blackpill memerlukan ST-Link V2 (SWD) karena tidak memiliki chip USB-to-Serial, sedangkan ESP32 dapat diprogram langsung lewat USB berkat chip CP2102. Program *Blink* berhasil dijalankan pada kedua papan, dengan Arduino pada Blackpill (LED PC13 aktif *LOW*) dan ESP-IDF pada ESP32. ESP-IDF memakai `app_main()` dan `vTaskDelay()` berbasis FreeRTOS, sehingga memberi kendali lebih besar dengan abstraksi lebih rendah dibanding Arduino.

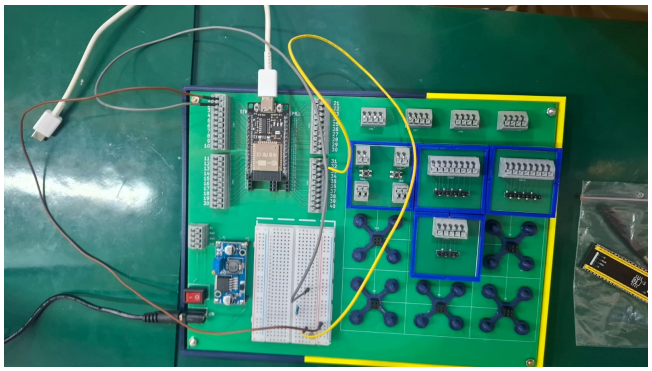
Pada pembacaan GPIO, resistor *pull-up* dan *pull-down*, baik eksternal maupun internal, mencegah pin *floating* dan menghasilkan pembacaan yang sama secara logika. Resistor internal ESP32 lebih lemah (sekitar 45 kΩ) dan tidak tersedia pada GPIO 34–39. *Debouncing* dengan `millis()` menjaga satu penekanan terbaca satu kali, dengan waktu *debounce* 50 ms sebagai kompromi antara bersihnya sinyal dan responsifnya tombol. Pada masukan analog, ADC 12 bit ESP32 membaca perubahan tegangan pembagi LDR dalam 4096 tingkat, dan nilainya turun dari sekitar 4030 ke sekitar 2730 seiring berkurangnya cahaya, sesuai teori pembagi tegangan.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 16: Dokumentasi 1



Gambar 17: Dokumentasi 2